PROYECTO DE COMPILADORES

Analizador sintáctico y árbol de sintaxis abstracta

Flex, Bison, CLion, Graphviz y Git

Documentación completa del desarrollo del proyecto

Autores

Hernández Ibarra Ian

Martínez Velázquez Zuly Meyai

Hernández Cruz José Alberto

16 de octubre de 2026

Repositorio: github.com/ian43256/Creaci-n-de-un-compilador

1. Introducción

Este documento resume todo el trabajo realizado para construir, desde cero, un pequeño analizador de expresiones aritméticas que genera un **árbol de sintaxis abstracta (AST)** usando **Flex** y **Bison** dentro de **CLion**, lo dibuja con **Graphviz** y se desarrolla en equipo mediante **Git** y **GitHub**.

El programa recibe una expresión como `2+3*(4-1)`, comprueba que esté bien formada, construye su árbol, lo imprime en consola y genera una imagen PNG del árbol.

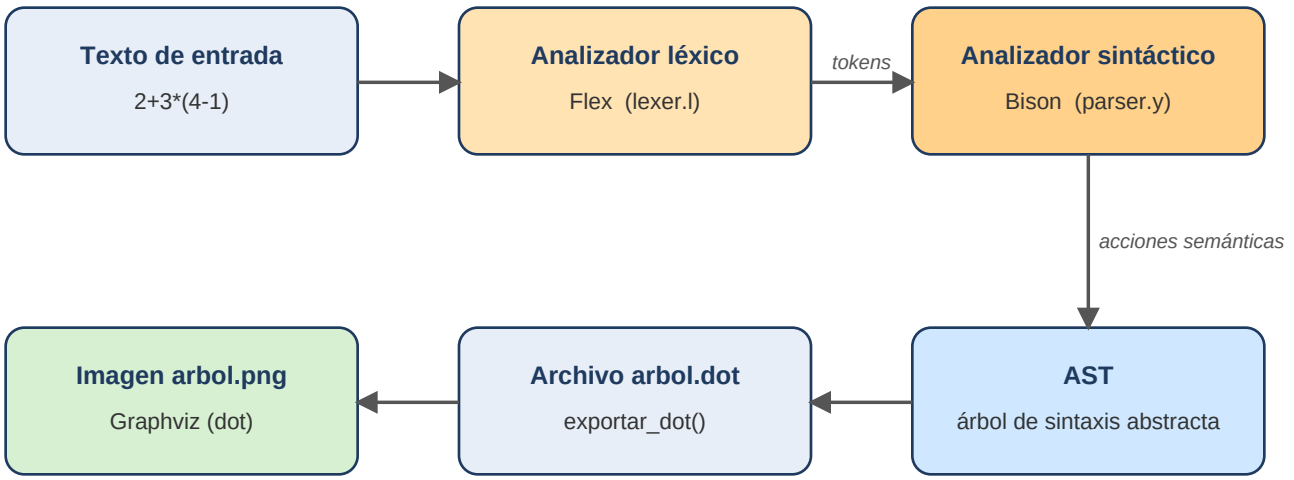


Figura 1. Flujo completo del programa, desde el texto de entrada hasta la imagen del árbol.

2. ¿Qué hace un analizador sintáctico?

Un compilador trabaja por etapas. El **analizador sintáctico** (*parser*) es la segunda: recibe la secuencia de **tokens** que produce el analizador léxico y verifica si esa secuencia respeta las reglas de la **gramática** del lenguaje. Si las respeta, construye una representación estructurada del programa; si no, informa un error de sintaxis.

2.1 Sus funciones principales

- **Verificar la estructura.** Comprueba que los tokens estén en un orden válido. `2+*3` tiene tokens correctos, pero su orden no forma una expresión válida.
- **Aplicar una gramática libre de contexto.** Las reglas describen cómo se forman las construcciones del lenguaje (por ejemplo, una expresión es un número, o dos expresiones unidas por un operador).
- **Construir una estructura jerárquica.** Produce un árbol de análisis o un AST que captura la agrupación y la precedencia de los operadores.
- **Detectar y reportar errores sintácticos**, indicando qué token encontró y cuáles esperaba.
- **Servir a las etapas siguientes.** El análisis semántico, la generación de código y la optimización trabajan sobre el árbol, no sobre el texto original.

2.2 Diferencia con el analizador léxico

	Analizador léxico (Flex)	Analizador sintáctico (Bison)
Entrada	Caracteres del texto fuente	Secuencia de tokens
Salida	Tokens: NUMERO, '+', '(' ...	Árbol de análisis o AST
Herramienta formal	Expresiones regulares y autómatas finitos	Gramáticas libres de contexto y autómatas de pila
Pregunta que responde	¿Qué palabras o símbolos hay?	¿Están bien ordenados y cómo se agrupan?
Archivo del proyecto	`lexer.l`	`parser.y`

2.3 Cómo trabaja Bison

Bison genera un analizador **ascendente (bottom-up)** de tipo **LALR(1)**: lee los tokens de izquierda a derecha, con un token de anticipación, y usa una pila. Sus dos operaciones básicas son:

- **Desplazar (shift):** pasar el siguiente token a la pila.
- **Reducir (reduce):** cuando los elementos de la cima de la pila coinciden con el lado derecho de una regla, reemplazarlos por el símbolo del lado izquierdo y ejecutar la **acción semántica** de esa regla.

En este proyecto, cada acción semántica crea un nodo del árbol. Como las reducciones ocurren de abajo hacia arriba, el árbol se construye desde las hojas hasta la raíz. Las declaraciones `%left` fijan la **precedencia y asociatividad**: la multiplicación y la división se agrupan antes que la suma y la resta, y se evalúan de izquierda a derecha.

2.4 La gramática del proyecto

```
programa  : expresion
expresion : NUMERO
          | expresion '+' expresion
          | expresion '-' expresion
          | expresion '*' expresion
          | expresion '/' expresion
          | '(' expresion ')'
```

2.5 Seguimiento del análisis de 2+3*(4-1)

La tabla muestra cómo Bison desplaza y reduce. *expr* abrevia *expresion*. Cada reducción marcada con "nodo" o "hoja" ejecuta una acción que crea un nodo del AST.

#	Pila	Entrada restante	Acción
1	(vacía)	2 + 3 * (4 - 1)	Desplazar NUMERO(2)
2	NUMERO	+ 3 * (4 - 1)	Reducir a expr: hoja 2
3	expr	+ 3 * (4 - 1)	Desplazar '+'
4	expr +	3 * (4 - 1)	Desplazar NUMERO(3)
5	expr + NUMERO	* (4 - 1)	Reducir a expr: hoja 3
6	expr + expr	* (4 - 1)	Desplazar '*' (precedencia mayor que '+')
7	expr + expr *	(4 - 1)	Desplazar '('
8	expr + expr * (	4 - 1)	Desplazar NUMERO(4)

#	Pila	Entrada restante	Acción
9	... (NUMERO	- 1)	Reducir a expr: hoja 4
10	... (expr	- 1)	Desplazar '-'
11	... (expr -	1)	Desplazar NUMERO(1)
12	... (expr - NUMERO	)	Reducir a expr: hoja 1
13	... (expr - expr	)	Reducir: nodo '-' con hijos 4 y 1
14	... (expr	)	Desplazar ')'
15	... (expr)	(fin)	Reducir: los paréntesis solo agrupan, no crean nodo
16	expr + expr * expr	(fin)	Reducir: nodo '*' con hijos 3 y (4-1)
17	expr + expr	(fin)	Reducir: nodo '+' con hijos 2 y (3*(4-1))
18	expr	(fin)	Reducir a programa: raiz = nodo '+'. Aceptar.

El resultado es el siguiente árbol. Cada operador es un nodo interno y cada número es una hoja. Por la precedencia, la multiplicación queda debajo de la suma y la resta, la más profunda, queda debajo de la multiplicación.

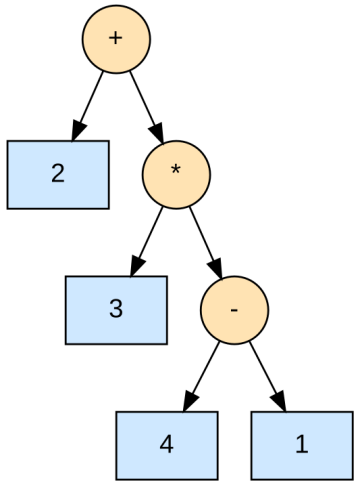


Figura 2. AST de 2+3*(4-1) generado por el programa con Graphviz (círculos: operadores, cajas: números).

3. Herramientas y entorno

Herramienta	Para qué se usa	Detalle de instalación
CLion	IDE donde se escribe, compila y ejecuta el proyecto. Trae gcc (MinGW), CMake y depurador.	Instalador de JetBrains; licencia de prueba o de estudiante.
MSYS2	Medio para instalar Flex y Bison en Windows.	Instalar en `C:\msys64`.
Flex 2.6.4	Genera el analizador léxico a partir de `lexer.l`.	`pacman -S flex bison`
Bison 3.8.2	Genera el analizador sintáctico a partir de `parser.y`.	Se instala con el mismo comando.
CMake	Orquesta la compilación: ejecuta Bison y Flex y compila el resultado.	Viene incluido en CLion (Bundled).
Graphviz 16.1.0	Dibuja el árbol a partir de un archivo DOT.	Instalador de 64 bits marcando "Add Graphviz to the system PATH".
Git y GitHub	Control de versiones y trabajo en equipo.	git-scm.com y cuenta de GitHub.

Nota importante. Flex y Bison **no** existen en MSYS2 con el prefijo `mingw-w64-x86_64-`. Se instalan como paquetes normales de MSYS2 (`pacman -S flex bison`) y quedan en `C:\msys64\usr\bin`. Por eso el `CMakeLists.txt` agrega esa ruta a la búsqueda de programas.

Estructura del proyecto

Archivo	Función
`CMakeLists.txt`	Define cómo se generan y compilan el parser, el lexer y los ejecutables.
`parser.y`	Gramática (Bison), definición del nodo, construcción del AST, impresión y exportación a DOT.
`lexer.l`	Reglas léxicas (Flex) que reconocen números, operadores y paréntesis.
`main.c`	Llama al parser, imprime el árbol, genera arbol.dot y arbol.png.
`lexers/`	Carpeta con un lexer independiente por integrante; cada .l se compila como su propio programa.
`.gitignore`	Evita subir `cmake-build-debug/` y `.idea/`.

4. Código final del proyecto

Estos son los cuatro archivos principales en su versión final.

`parser.y`

Define el nodo del árbol y la gramática. Usa `%code requires` para que la definición de `ASTNode` quede en `parser.tab.h`, y `%code` para el código que debe ir después de esa definición.

```
%code requires {
    typedef enum { N_NUMERO, N_SUMA, N_RESTA, N_MULT, N_DIV } NodeType;

    typedef struct ASTNode {
        NodeType tipo;
        int valor;
        struct ASTNode *izq;
        struct ASTNode *der;
    } ASTNode;

    ASTNode* crear_nodo_numero(int v);
    ASTNode* crear_nodo_operacion(NodeType t, ASTNode* i, ASTNode* d);
    void imprimir_arbol(ASTNode* nodo, int nivel);
    void exportar_dot(ASTNode* raiz, const char* archivo);
    void liberar_arbol(ASTNode* nodo);
}

%code {
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);

ASTNode* raiz = NULL;
}

%union {
    int val_int;
    struct ASTNode* nodo;
}

%token <val_int> NUMERO
%type <nodo> expresion programa

%left '+' '-'
%left '*' '/'

%%

programa:
    expresion { raiz = $1; $$ = $1; }
    ;

expresion:
      NUMERO                { $$ = crear_nodo_numero($1); }
    | expresion '+' expresion { $$ = crear_nodo_operacion(N_SUMA,  $1, $3); }
    | expresion '-' expresion { $$ = crear_nodo_operacion(N_RESTA, $1, $3); }
    | expresion '*' expresion { $$ = crear_nodo_operacion(N_MULT,  $1, $3); }
    | expresion '/' expresion { $$ = crear_nodo_operacion(N_DIV,   $1, $3); }
    | '(' expresion ')'       { $$ = $2; }
    ;

%%

ASTNode* crear_nodo_numero(int v) {
    ASTNode* n = (ASTNode*)malloc(sizeof(ASTNode));
    n->tipo = N_NUMERO;
    n->valor = v;
    n->izq = n->der = NULL;
    return n;
```

```
}

ASTNode* crear_nodo_operacion(NodeType t, ASTNode* i, ASTNode* d) {
    ASTNode* n = (ASTNode*)malloc(sizeof(ASTNode));
    n->tipo = t;
    n->valor = 0;
    n->izq = i;
    n->der = d;
    return n;
}

void imprimir_arbol(ASTNode* nodo, int nivel) {
    if (nodo == NULL) return;
    for (int i = 0; i < nivel; i++) printf("  ");

    if (nodo->tipo == N_NUMERO) {
        printf("Numero: %d\n", nodo->valor);
    } else {
        printf("Operacion: ");
        switch (nodo->tipo) {
            case N_SUMA:  printf("+\n"); break;
            case N_RESTA: printf("-\n"); break;
            case N_MULT:  printf("*\n"); break;
            case N_DIV:   printf("/\n"); break;
            default: break;
        }
        imprimir_arbol(nodo->izq, nivel + 1);
        imprimir_arbol(nodo->der, nivel + 1);
    }
}

/* ---------- Exportar el arbol a formato DOT (Graphviz) ---------- */
static int contador_id = 0;

static int dot_nodo(ASTNode* n, FILE* f) {
    int id = contador_id++;

    if (n->tipo == N_NUMERO) {
        fprintf(f, "  n%d [label=\"%d\", shape=box, style=filled, fillcolor=\"#cfe8ff\"];\n",
                id, n->valor);
    } else {
        const char* op = "?";
        switch (n->tipo) {
            case N_SUMA:  op = "+"; break;
            case N_RESTA: op = "-"; break;
            case N_MULT:  op = "*"; break;
            case N_DIV:   op = "/"; break;
            default: break;
        }
        fprintf(f, "  n%d [label=\"%s\", shape=circle, style=filled, fillcolor=\"#ffe3b3\"];\n",
                id, op);

        if (n->izq) {
            int h = dot_nodo(n->izq, f);
            fprintf(f, "  n%d -> n%d;\n", id, h);
        }
        if (n->der) {
            int h = dot_nodo(n->der, f);
            fprintf(f, "  n%d -> n%d;\n", id, h);
        }
    }
    return id;
}

void exportar_dot(ASTNode* raiz, const char* archivo) {
    FILE* f = fopen(archivo, "w");
    if (f == NULL) {
        fprintf(stderr, "No se pudo crear %s\n", archivo);
        return;
    }
    contador_id = 0;
    fprintf(f, "digraph AST {\n");
```

```
    fprintf(f, "  node [fontname=\"Arial\", fontsize=14];\n");
    if (raiz != NULL) dot_nodo(raiz, f);
    fprintf(f, "}\n");
    fclose(f);
}

void liberar_arbol(ASTNode* nodo) {
    if (nodo == NULL) return;
    liberar_arbol(nodo->izq);
    liberar_arbol(nodo->der);
    free(nodo);
}

void yyerror(const char *s) {
    fprintf(stderr, "Error sintactico: %s\n", s);
}
```

`lexer.l`

Reconoce los tokens y se los entrega al parser. El número viaja en `yylval.val_int`, definido en el `%union`. El salto de línea devuelve 0, que Bison interpreta como fin de entrada.

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "parser.tab.h"
%}

%option noyywrap

%%

[0-9]+      { yylval.val_int = atoi(yytext); return NUMERO; }
[ \t\r]+    { /* ignorar espacios */ }
\n          { return 0; }
[-+*/()]    { return yytext[0]; }
.           { fprintf(stderr, "Caracter invalido: %s\n", yytext); }

%%
```

`main.c`

Ejecuta el análisis, imprime el árbol, escribe `arbol.dot` y llama a `dot` para crear el PNG.

```
#include <stdio.h>
#include <stdlib.h>
#include "parser.tab.h"

extern ASTNode* raiz;
int yyparse(void);

int main(void) {
    printf("Escribe una expresion (ej. 2+3*(4-1)) y presiona Enter:\n");
    if (yyparse() == 0) {
        printf("\nArbol de sintaxis abstracta:\n");
        imprimir_arbol(raiz, 0);

        exportar_dot(raiz, "arbol.dot");
        int r = system("dot -Tpng arbol.dot -o arbol.png");
        if (r == 0) {
            printf("\nImagen generada: arbol.png\n");
        } else {
            printf("\nNo se pudo generar el PNG. Verifica que Graphviz este instalado (dot -V).\n");
        }

        liberar_arbol(raiz);
    }
    return 0;
}
```

`CMakeLists.txt`

Le indica a CMake que ejecute Bison y Flex, que compile los archivos generados con `main.c` y que cree un ejecutable por cada lexer de la carpeta `lexers/`.

```
cmake_minimum_required(VERSION 3.15)
project(ast_bison C)

set(CMAKE_C_STANDARD 11)

# Rutas comunes donde se instalan Flex y Bison en Windows
if(WIN32)
    list(APPEND CMAKE_PROGRAM_PATH "C:/msys64/usr/bin" "C:/winflexbison")
endif()

find_package(BISON REQUIRED)
find_package(FLEX REQUIRED)

include_directories(${CMAKE_CURRENT_BINARY_DIR})

# Programa principal: AST con Flex + Bison
BISON_TARGET(Parser parser.y ${CMAKE_CURRENT_BINARY_DIR}/parser.tab.c
             DEFINES_FILE ${CMAKE_CURRENT_BINARY_DIR}/parser.tab.h)
FLEX_TARGET(Lexer lexer.l ${CMAKE_CURRENT_BINARY_DIR}/lexer.yy.c)
ADD_FLEX_BISON_DEPENDENCY(Lexer Parser)

add_executable(ast_bison
        main.c
        ${BISON_Parser_OUTPUTS}
        ${FLEX_Lexer_OUTPUTS})

# Un ejecutable por cada lexer dentro de lexers/ (cada .l incluye su propio main)
file(GLOB LEXER_FILES CONFIGURE_DEPENDS ${CMAKE_SOURCE_DIR}/lexers/*.l)
foreach(LEXER_FILE ${LEXER_FILES})
    get_filename_component(LEXER_NAME ${LEXER_FILE} NAME_WE)
    FLEX_TARGET(${LEXER_NAME} ${LEXER_FILE} ${CMAKE_CURRENT_BINARY_DIR}/${LEXER_NAME}.yy.c)
    add_executable(${LEXER_NAME} ${FLEX_${LEXER_NAME}_OUTPUTS})
endforeach()
```

5. Compilación, ejecución y resultado

En CLion: **File > Reload CMake Project**, luego **Build** y **Run** con el ejecutable `ast_bison`. Al escribir `2+3*(4-1)` el programa imprime:

```
Arbol de sintaxis abstracta:
Operacion: +
  Numero: 2
  Operacion: *
    Numero: 3
    Operacion: -
      Numero: 4
      Numero: 1

Imagen generada: arbol.png
```

La indentación refleja la profundidad de cada nodo. El PNG se crea en la carpeta de trabajo del ejecutable (`cmake-build-debug`) y coincide con la Figura 2.

Cada lexer de `lexers/` se prueba por separado: se elige su ejecutable en el selector de CLion, se ejecuta, se escribe un texto y se termina la entrada con `Ctrl+Z` y Enter en Windows. Por ejemplo, `lexer_ejemplo` con `12+3` imprime NUMERO: 12, OPERADOR: + y NUMERO: 3.

6. Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
`target not found: mingw-w64-x86_64-flex`	En MSYS2 Flex y Bison no llevan ese prefijo.	`pacman -S flex bison` y apuntar CMake a `C:\msys64\usr\bin`.
`unknown type name 'ASTNode'` al compilar parser.y	El bloque `%{ %}` se copia al inicio del .c, antes de definir el tipo.	Cambiarlo por `%code { }`, que se inserta después de `%code requires`.
Errores de C como `expected identifier before '%' token` en main.c	Se pegó el código de parser.y dentro de main.c.	Cada archivo tiene su contenido: parser.y empieza con `%code requires` y main.c con `#include`.
`dot` no se reconoce en la terminal	CLion lee el PATH solo al iniciar.	Cerrar CLion por completo y abrirlo de nuevo tras instalar Graphviz.
`git push` rechazado (`fetch first`)	El repositorio de GitHub ya tenía archivos.	`git pull origin main --allow-unrelated-histories`, guardar el mensaje del merge en vim (`:wq`) y repetir el push.
Archivos nuevos en el proyecto equivocado	Se trabajaba en un proyecto antiguo con otro código.	Crear un proyecto nuevo de tipo C Executable (`ast_bison`).

7. Git y GitHub: trabajo en equipo

El proyecto se publicó en el repositorio `github.com/ian43256/Creaci-n-de-un-compilador`. Las dos personas restantes se agregaron como colaboradoras (Settings > Collaborators) y aceptaron la invitación.

7.1 Primera publicación

```
git init
git add .
git commit -m "AST con Flex y Bison"
git branch -M main
git remote add origin https://github.com/ian43256/Creaci-n-de-un-compilador.git
git pull origin main --allow-unrelated-histories
git push -u origin main
```

7.2 Flujo de cada integrante

Paso	Qué hacer en CLion o en la terminal
1. Clonar (una vez)	File > New > Project from Version Control; pegar la URL del repositorio; Clone; Trust Project.
2. Identidad (una vez)	`git config --global user.name "Nombre"` y `git config --global user.email "correo-de-GitHub"`. El correo debe ser el de su cuenta para que los commits se le atribuyan.
3. Crear rama	Clic en el nombre de la rama > New Branch, o `git checkout -b nombre-rama`.
4. Commit	Ctrl+K, o `git add archivo` y `git commit -m "mensaje"`. Varios commits pequeños en lugar de uno grande.
5. Push	Ctrl+Shift+K, o `git push -u origin nombre-rama` la primera vez.
6. Actualizar	Git > Pull, o `git pull`, antes de empezar a trabajar.
7. Pull Request	En GitHub: Compare & pull request > Create pull request. El dueño del repositorio hace Merge.

Como cada integrante edita archivos distintos, las ramas se fusionan sin conflictos. Las contribuciones se pueden revisar en GitHub, en **Insights > Contributors** y en la pestaña **Commits** de cada rama.

8. Aportaciones asignadas al equipo

Integrante	Rama	Aportación
Hernández Ibarra Ian	main / grafica-arbol	Proyecto base (parser, lexer, main, CMake), exportación del AST a Graphviz, repositorio, carpeta `lexers/` y plantilla.
Martínez Velázquez Zuly Meyai	lexer-columnas-zuly	Mejora de `lexer.l` para reportar la columna del carácter inválido y lexer `lexers/lexer2_1.l`.
Hernández Cruz José Alberto	errores-alberto	Mensajes de error detallados en `parser.y`, aviso de expresión inválida en `main.c` y lexer `lexers/lexer3_1.l`.

8.1 Zuly: errores léxicos con columna

Dos variables llevan la columna actual. `YY_USER_ACTION` es una macro de Flex que se ejecuta antes de cada regla: guarda dónde empieza el token y avanza la columna según su longitud (`yyleng`). Así, el error indica **dónde** está el carácter inválido. Con 2+a se espera *Error lexico: caracter invalido 'a' en la columna 3*.

lexer.l (versión de Zuly)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "parser.tab.h"

static int columna = 1;
static int col_inicio = 1;

#define YY_USER_ACTION col_inicio = columna; columna += yyleng;
%}

%option noyywrap

%%

[0-9]+      { yylval.val_int = atoi(yytext); return NUMERO; }
[ \t\r]+    { /* ignorar espacios */ }
\n          { columna = 1; return 0; }
[-+*/()]    { return yytext[0]; }
.           { fprintf(stderr, "Error lexico: caracter invalido '%s' en la columna %d\n",
                      yytext, col_inicio); }

%%
```

lexers/lexer2_1.l (palabras reservadas e identificadores)

```
%{
#include <stdio.h>
%}

%option noyywrap

%%

"if"|"else"|"while"|"for"|"int"|"float"|"return"    { printf("PALABRA_RESERVADA: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]*                              { printf("IDENTIFICADOR: %s\n", yytext); }
[0-9]+                                              { printf("NUMERO: %s\n", yytext); }
[-+*/=(){};,]                                       { printf("SIMBOLO: %s\n", yytext); }
[ \t\r\n]+                                          { /* ignorar espacios */ }
.                                                   { printf("INVALIDO: %s\n", yytext); }

%%

int main(void) {
    yylex();
    return 0;
}
```

Con `if x1 + 25` imprime PALABRA_RESERVADA: if, IDENTIFICADOR: x1, SIMBOLO: + y NUMERO: 25. La regla de las palabras reservadas va antes que la de identificadores: cuando dos reglas coinciden con la misma longitud, Flex elige la primera.

8.2 Alberto: errores de sintaxis claros y fallo del análisis

En `parser.y` se agrega una línea justo antes de `%union`:

```
%define parse.error verbose
```

Con ella, `2+*3` deja de mostrar solo *syntax error* y pasa a indicar el token inesperado y los esperados, por ejemplo *unexpected '*', expecting NUMERO or '('*.

main.c (versión de Alberto)

Si `yyparse()` falla, avisa que no se generó el árbol y termina con código 1, la convención en C para indicar error (0 significa éxito).

```
#include <stdio.h>
#include <stdlib.h>
#include "parser.tab.h"

extern ASTNode* raiz;
int yyparse(void);

int main(void) {
    printf("Escribe una expresion (ej. 2+3*(4-1)) y presiona Enter:\n");

    if (yyparse() != 0) {
        printf("\nLa expresion no es valida, no se genero el arbol.\n");
        return 1;
    }

    printf("\nArbol de sintaxis abstracta:\n");
    imprimir_arbol(raiz, 0);

    exportar_dot(raiz, "arbol.dot");
    int r = system("dot -Tpng arbol.dot -o arbol.png");
    if (r == 0) {
        printf("\nImagen generada: arbol.png\n");
    } else {
        printf("\nNo se pudo generar el PNG. Verifica que Graphviz este instalado (dot -V).\n");
    }

    liberar_arbol(raiz);
    return 0;
}
```

lexers/lexer3_1.l (enteros y decimales)

```
%{
#include <stdio.h>
%}

%option noyywrap

%%

[0-9]+\.[0-9]+      { printf("DECIMAL: %s\n", yytext); }
[0-9]+              { printf("ENTERO: %s\n", yytext); }
[-+*/()]            { printf("OPERADOR: %s\n", yytext); }
[ \t\r\n]+          { /* ignorar espacios */ }
.                   { printf("INVALIDO: %s\n", yytext); }

%%

int main(void) {
    yylex();
    return 0;
}
```

Con `3.14 + 7` imprime DECIMAL: 3.14, OPERADOR: + y ENTERO: 7. La regla del decimal va primero porque, si no, `3.14` se partiría en un entero, un punto inválido y otro entero.

9. Conclusiones

- El analizador léxico y el sintáctico se complementan: Flex convierte el texto en tokens y Bison valida su estructura con una gramática y construye el AST mediante acciones semánticas.
- Declarar la precedencia con `%left` evita ambigüedades y hace que el árbol respete el orden de las operaciones.
- El AST es la base para etapas posteriores del compilador, como evaluar la expresión, comprobar tipos o generar código.
- Graphviz permite visualizar el árbol y comprobar de forma inmediata que el análisis es correcto.
- Git y las ramas individuales permiten repartir el trabajo sin conflictos y dejar registro de la participación de cada integrante.